

Overview

Welcome to Semantica

Core Concepts

Core Concepts

Modules

Choose the Right Module

Glossary

Guides

Architecture

Learning More

GraphRAG: Graph-Augmented Retrieval

Ingest Anything

LLM Integrations

Pipeline Builder

Multi-Agent Systems

Decision Intelligence

Context Graphs

Agent Memory

Provenance & Audit Trails

Semantic Extraction

Ontology Management

Reasoning & Rules

Export & Serialization

Deduplication & Entity Merging

MCP Server

SHACL Validation

Conflict Detection & Resolution

Change Management & Versioning

Policy Engine

Visualization

Distance Intelligence

Graph Analytics

Integrations

Agno Integration

CrewAI Integration

LangChain Integration

Docling Integration

Snowflake Integration

Databricks Integration

Salesforce Integration

Graph Stores

Apache AGE Graph Store

Vector Stores

pgvector Store

FAQ

FAQ

Community

Community

Community Projects

Contributing

Governance

Citation

License

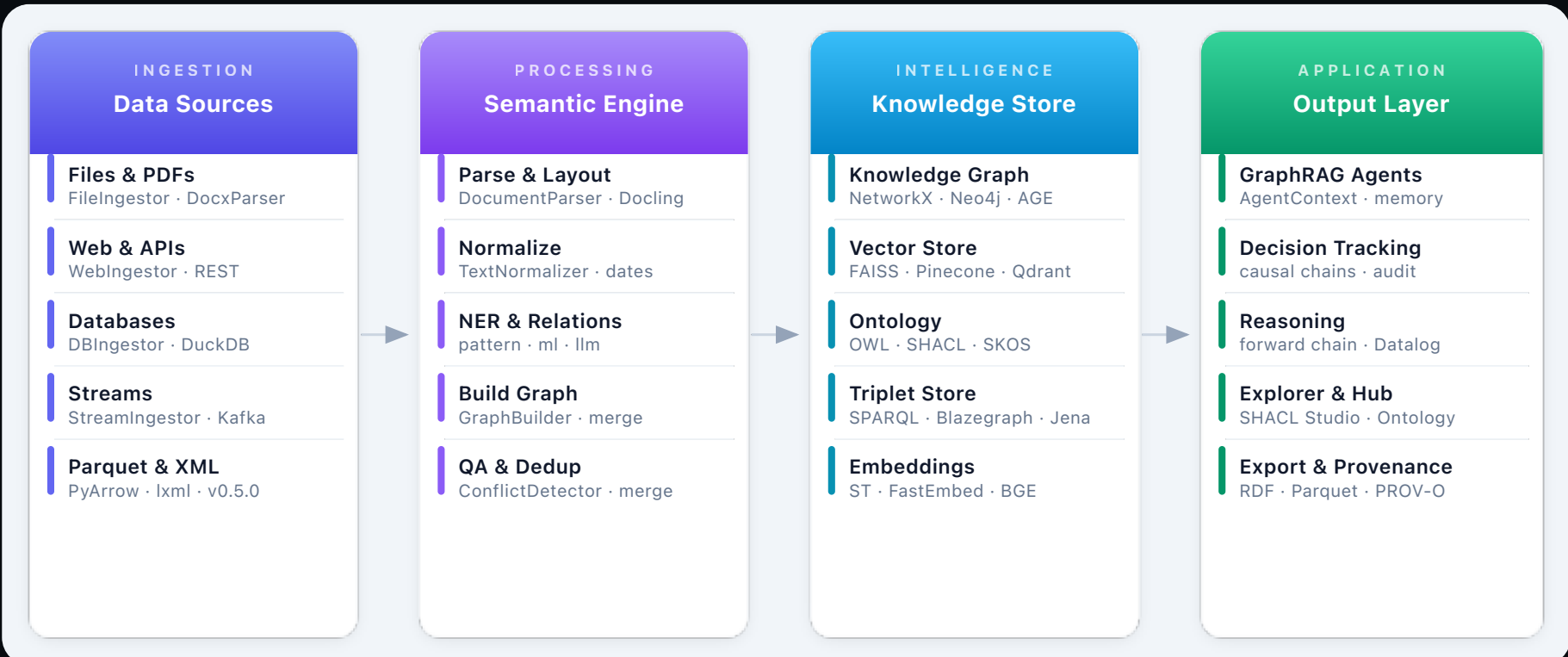
Guides

Architecture

Four-layer, modular architecture designed for independent component use, clean separation of concerns, and full extensibility.

Semantica is built around a four-layer modular architecture. Import only what you need: the framework never forces a full stack. Every component is independently swappable, and every layer communicates through clean interfaces with no hidden coupling.

Four-Layer Architecture



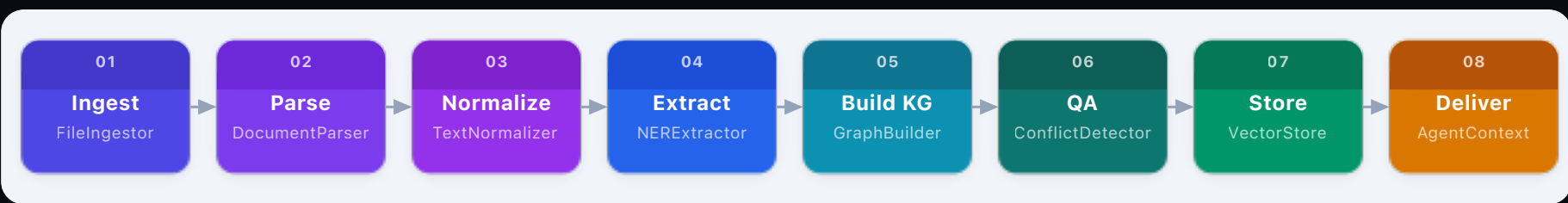
Layer 1: Ingestion Layer 2: Processing Layer 3: Intelligence Layer 4: Application

Persistent knowledge stores and embedding infrastructure that power retrieval and reasoning.

COMPONENT	MODULE	DESCRIPTION
Knowledge Graph	kg	Graph construction, temporal models, analytics, Distance Intelligence
Vector Store	vector_store	pgvector, Qdrant, Weaviate, Pinecone: semantic similarity search
Ontology	ontology	OWL/RDFS modeling, SHACL validation, ontology alignment
Triplet Store	triplet_store	RDF triple storage and SPARQL querying
Embeddings	embeddings	Sentence-Transformers, FastEmbed, OpenAI, BGE
Temporal	kg.TemporalKnowled geGraph	valid_from / valid_until , Allen interval algebra (v0.4.0)

Data Flow

Every pipeline follows the same linear path from raw source to delivered output:



Module Map

LAYER	CATEGORY	MODULES
Layer 1: Ingestion	Sources	ingest , split
Layer 2: Processing	Transform	parse , normalize , semantic_extract , deduplication , conflicts
Layer 3: Intelligence	Stores	kg , vector_store , graph_store , triplet_store , embeddings , ontology
Layer 4: Application	Delivery	context , reasoning , export , visualization , explorer , pipeline
:	Cross-cutting	provenance , change_management , llms , mcp_server , seed , evals , core , utils

Extension Points

Every layer exposes a registry-based extension point. Register custom implementations and they participate in the full pipeline with zero changes to core code.

```
Custom Ingestor Custom Extractor Custom Plugin

from semantica.ingest.registry import method_registry

def custom_file_ingestor(source):
    # Return a list of document dicts with 'text', 'metadata', 'source'
    return [{ "text": "...", "metadata": {}, "source": source}]

# Register under the "file" task category with a unique name
method_registry.register("file", "my_custom_format", custom_file_ingestor)

available = method_registry.list_all("file")
```

Design Decisions

- ▼  **Modularity: use only what you need**
Every component works standalone. `NERExtractor` runs without a graph store. `VectorStore` runs without decision tracking. The framework never forces a full stack instantiation; you pay only for what you import.
- ▼  **Pluggability: extend without modifying core**
Custom ingestors, extractors, validators, and exporters follow the same base class pattern. Register them via `PluginRegistry` and they participate in the full pipeline (provenance tracking, retry policies, and parallel execution included) with no changes to core code.
- ▼  **Provenance by default**
Lineage tracking is built into graph construction at the lowest level. Every node and edge carries a `source_id` pointing back to the originating document, extraction method, and timestamp. There is no opt-in required; provenance is always on.
- ▼  **Configuration over convention**
Centralized `ConfigManager` with environment variable overrides. No magic defaults; all behavior is explicit and overridable. Suitable for multi-environment deployments where dev, staging, and production need different backends.

Performance Characteristics

CHARACTERISTIC	MECHANISM
Parallel execution	<code>Pipeline(workers=N)</code> with configurable workers per stage
Delta processing	Incremental graph updates (no full recompute on new data)
Streaming ingestion	Process large corpora without loading everything into memory
Backend flexibility	Swap in-memory NetworkX for Neo4j / FalkorDB with no API changes
Deduplication v2	<code>blocking_v2</code> , <code>hybrid_v2</code> , <code>semantic_v2</code> : up to 7x faster than v1
Indexed search	Explorer search at 0.004ms on 118k nodes (v0.5.0)

- [Modules](#): full module documentation with code examples.
- [Learning More](#): configuration reference, performance guide, and troubleshooting.
- [Pipeline Reference](#): pipeline orchestration, workers, and retry policies.
- [Core Reference](#): framework lifecycle, plugin registry, and configuration.

< Glossary

Learning More >

On this page

Four-Layer Architecture

Data Flow

Module Map

Extension Points

Design Decisions

Performance Characteristics

